REST API Specification Design

Version 1.0

Version Date: 11/18/2024

Author: MyAccess Team

CVS Health
One CVS Drive
Woonsocket, RI 02895

Table of Contents

1 Document Overview

This document provides a thorough specification design for the RESTful API that will serve as the backbone for REST-based applications. The purpose of these REST APIs is to expose user and access management-related services as endpoints that can be configured on applications like SailPoint Identity Security Cloud. Note that this is a generic, high-level specification that can be used as a reference when designing the application-specific REST API specifications for various apps, and not the actual REST API Specification for a particular application.

1.1 Scope

This specification covers all aspects of the API design, including:

1. **Endpoint Definitions**: Detailed descriptions of each endpoint, including URL paths, HTTP methods, and expected requests and responses.
2. **Authentication and Authorization**: Guidelines and mechanisms for securing the API, including token-based authentication and role-based access control.
3. **Data Models**: Definitions of the data structures used in requests and responses, including field types and validation rules.
4. **Error Handling**: Standardized error codes and messages to confirm consistent and informative error reporting.
5. **Rate Limiting**: Policies to prevent abuse and confirm fair usage of the API resources.
6. **Versioning**: Strategy for versioning the API to manage updates and backward compatibility.

1.2 Audience

This document is intended for:

- **Application Technical Owners**: Who will implement the REST API based on the specifications outlined herein or integrate client applications with the endpoints.

1.3 Application Categorization

Application Category	Category Description
Centrally-Managed Application Refer Section 4.1	An application is considered as a centrally managed application when the user and entitlements are managed through a centralized directory (such as Active Directory, Azure AD or other LDAP applications, RACF, TOP SECRET).
Locally-Managed Application Refer Section 4.2	An application is considered as a locally managed application when the user and entitlements are managed locally by in-house application teams or vendor managed databases.
Hybrid Application Refer Section 4.3	An application is considered as a hybrid application when there is a component of user and entitlements managed through a centralized directory (such as Active Directory, Azure AD or other LDAP applications, RACF, TOP SECRET). Additionally, there is another component where the user and entitlements are managed locally within the application. Such applications have a “hybrid” approach to their user & access management.

2 Requirements

2.1 Functional Requirements

Functional Requirements:

Req #	Requirement
Req 1	The system must provide REST API endpoints as specified in Section 4.1 for Centrally Managed applications for MyAccess to execute the following operations: 1. Get Entitlements Metadata 2. Get Entitlements Count
Req 2	The system must provide REST API endpoints as specified in Section 4.2 for Locally Managed applications for MyAccess to execute the following operations: 1. Get Entitlements Metadata 2. Get Entitlements Count 3. Get Users Membership Metadata 4. Get Single User Membership Metadata 5. Revoke User Membership
Req 3	The system must provide REST API endpoints as specified in Section 4.3 for Hybrid applications for MyAccess to execute the following operations: 1. Get Entitlements Metadata (Centrally Managed) 2. Get Entitlements Count (Centrally Managed) 3. Get Entitlements Metadata (Locally Managed) 4. Get Entitlements Count (Locally Managed) 5. Get Users Membership Metadata (Locally Managed) 6. Get Single User Membership Metadata (Locally Managed) 7. Revoke User Membership (Locally Managed)
Req 4	The system must be able to handle errors gracefully and return an appropriate error code and an error message to MyAccess based on the guidelines on error codes outlined in Section 3.4.
Req 5	The system must be able to support pagination when the volume of user or entitlement data being fetched isn't small enough to be returned in a single API call with reasonable latency.
Req 6	The system must be able to all enforce API Rate Limits on the server side as outlined in Section 3.2.

2.2 Non-Functional Requirements

Non-Functional Requirements:

Req #	Requirement
Req 1	The system must support an authentication mechanism for all the endpoints via one of the methods outlined in Section 3.5.
Req 2	The system must support at least one request daily from MyAccess for each of the REST API endpoint.

3 REST API Overview

> ***Note**: This is a generic, high-level specification that can be used as a reference for designing the application-specific REST API specifications for various apps, and not the actual REST API Specification for a particular application.*

REST (Representational State Transfer) APIs are a type of web service that allows interaction between different software applications over the internet. They leverage HTTP methods such as GET, POST, PUT, DELETE, and others to perform operations on resources represented in a standardized format, typically JSON.

REST APIs are particularly useful in scenarios where:

- **Interoperability**: Different systems, possibly developed in various languages, need to communicate, and exchange data.
- **Scalability**: The application needs to handle a large number of requests efficiently.
- **Stateless Operations**: Each request from a client contains all the information needed to process it, making the API stateless and scalable.
- **Resource-Oriented Architecture**: The application is designed around resources, which can be entities like users, files, or orders.

Using REST APIs typically involves:

1. **Making HTTP Requests**: Clients (like SailPoint ISC) send HTTP requests to specific endpoints defined by the API to perform various operations. Note that to ensure security of data in transit, REST APIs will require **HTTPS**.
2. **Handling HTTP Responses**: The application server processes the requests and sends back HTTP responses, including status codes and data in a standardized format.
3. **Authentication and Authorization**: Ensuring secure access to the API using methods such as OAuth tokens, API keys, or JWT (JSON Web Tokens).

3.1 Authentication Mechanism and Required Permissions

Authentication is a critical component of any REST API, confirming that only authorized users and systems can access the resources and perform operations. This section outlines the various authentication mechanisms supported by the API, including best practices for secure implementation.

The credentials used by service accounts must be rotated every 60 days. The application technical owner is accountable for the periodic rotation of credentials in collaboration with the IAM team to update the details on MyAccess.

3.1.1 Authentication Types

3.1.1.1 (Preferred) OAuth 2.0

OAuth 2.0 is an industry-standard protocol for authorization, which allows third-party applications to obtain limited access to a web service. Clients like SailPoint ISC obtain an access token via an authorization server. The token is then included in the API requests headers to indicate authenticated API calls.

3.1.1.2 Token-based Authentication

While more secure than basic authentication, a token-based authentication approach utilizes static tokens generated by a system. A token is passed either in the header or as part of the API body to authenticate the APIs.

3.1.1.3 (Least Preferred) Basic Authentication

This type of authentication can be less secure in comparison to some of the other authentication types. It utilizes a simple username & password-based authentication mechanism to authenticate an API call. A variation of this basic authentication is cookie-based authentication, where authentication takes place against a server using the usual username and password, and a cookie is sent back from the server. This cookie is then used in the API calls made to authenticate them.

3.1.2 Required Permissions

For a successful integration with SailPoint Identity Security Cloud, A Service Account with following permissions is typically required (i.e., full read and write permission)

- Read Accounts: Permission to read user account details.
- Write Accounts: Permission to create, update, or delete user account details.
- Read Entitlements: Permission to read entitlement details.
- Write Entitlements: Permission to assign or remove entitlements to/from user accounts.
- Read Groups: Permission to read group details.
- Write Groups: Permission to create, update, or delete groups.

3.2 IAM through REST APIs

IAM specific APIs are specialized RESTful services designed to handle user and access management within an organization. They provide functionalities such as:

- **User Provisioning**: Creating, updating, and deleting user accounts.
- **Access Management**: Assigning and revoking access to users based on roles or ad-hoc access requests.
- **Centralized View**: Aggregating user and access data into an IAM tool like SailPoint ISC.

3.2.1 SailPoint ISC Architecture

To securely communicate with CVS Health target systems, SailPoint uses Virtual Appliances, or VAs, to connect CVS Health tenant and on-premises applications. A VA is a Linux-based virtual machine that connects to CVS Health's sources and apps using SailPoint APIs, connectors, and integrations.

The virtual appliance is placed in a DMZ, such that it can send outbound traffic to the SailPoint ISC tenant and poll the cloud services. The VA hosts various connector implementations proprietary to SailPoint, like web services and JDBC.

3.3 Rate Limiting

Rate limiting is a crucial aspect of API management that controls the number of requests a client can make to the API within a specified time frame. This practice helps confirm the stability, security, and performance of the API. By implementing rate limiting, we protect the API from being overwhelmed by excessive requests, whether intentional or accidental.

3.3.1 Why Rate Limiting is Important

- **Resource Management:** APIs often interact with databases, services, and other resources. Without rate limiting, these resources can become overloaded, leading to degraded performance or downtime.
- **Fair Usage:** Rate limiting confirms that all clients have fair access to the API. It prevents any single client from monopolizing the API's resources, thus maintaining a balanced environment for all users.

- **Security:** Rate limiting helps mitigate the risk of abuse and attacks, such as Denial of Service (DoS) attacks. By capping the number of requests, we reduce the potential impact of malicious activities.
- **Cost Control:** For APIs that operate on a pay-as-you-go model or have limited computing resources, rate limiting helps manage costs by preventing excessive usage.
- **Improved User Experience:** By avoiding server overloads and confirming consistent performance, rate limiting contributes to a more reliable and smooth experience for all API consumers.

3.3.2 Rate Limiting Policy

Have a policy for rate limiting along the lines of **100 requests per 10 seconds** per client/access token. This means each client or access token can make up to **100 requests every 10 seconds**. If a client exceeds this limit, they can receive an error with a code like **429 Too Many Requests**, indicating that they should reduce or space out their request rate.

3.4 Pagination

Pagination is a technique used to divide a large set of results into discrete pages, making it easier to manage and navigate through the data. This practice helps enhance the performance of the API by reducing the amount of data transferred in a single response, thus improving response times, and reducing the load on both the server and the client.

Implementing pagination is essential to ensure efficient data retrieval and to prevent overloading the server with large requests. Clients are required to include pagination parameters in their requests to specify the desired page and the number of items per page. This allows for more efficient data handling and a better user experience by reducing the amount of data transferred in each request.

3.4.1 Why Pagination is Important

- **Performance Optimization**: By limiting the amount of data in each response, pagination helps improve the response time and reduces the load on both the server and the client.
- **Resource Management**: Handling smaller chunks of data reduces memory and processing requirements, which is particularly important for systems with limited resources.
- **User Experience**: Pagination enhances client experience by allowing them to navigate through manageable sets of data rather than being overwhelmed by a large dataset.
- **Scalability**: Pagination supports scalability by enabling the API to handle large datasets efficiently without compromising performance.

3.4.2 Pagination Strategies

A pagination strategy dictates how you expose a large dataset to the client in chunks, also known as pages. There are several different strategies for implementing pagination, each with its own advantages and use cases.

3.4.2.1 Caching Considerations

It is important to understand that the data during pagination is loaded in-memory and represents a point-in-time state of the data. The snapshot of the data is cached at the load of the first page. As a result, any updates that happen to the original data after the first page is fetched are not reflected in the API calls made in the duration in which the cache is not cleared. Consider a frequency to clear the cache after **2 minutes** of an idle state of the web services; i.e., when there has not been an API call for at least 2 minutes.

Below are some of the most commonly used pagination strategies.

3.4.2.2 Page Size Determination

To determine the maximum number of resources per page, consider the following factors:

1. **Payload Size**: Consider having a maximum capacity of 500KB of response. Accordingly, increase or decrease the number of resources returned in one API call.
2. **Response Time Per API Call**: Consider the maximum response time for an API call to be 250ms. If the average response is taking longer than 250ms, consider reducing the number of resources fetched. Similarly, if the response is rapid, you can consider increasing the page size to reduce the number of API calls needed.

Once you have identified the resource limit per page, ensure that your application can support returning objects equal to or less than the limit number. Set a default page size which is 50% of the limit identified. Allow for configuring page size at the time of the API call through query parameters.

3.4.2.2.1 Example: Benchmarking & Testing Page Limits

- Consider the size of one resource to be 2KB
- Let the initial page size be 250, so the payload size is 500KB
- Consider the API response time for 500 resources was 150ms.
- Increase page size to 300. Retest average response time
- Average response time is now 244ms, which is acceptable.

3.4.2.3 (Preferred) Offset and Limit based Pagination

Offset and limit-based pagination allows clients to request a specific subset of data from a larger collection. This technique is useful for improving performance and providing a better user experience when dealing with large datasets. The client specifies the starting point (offset) and the number of records to return (limit). Offset-based pagination allows for flexible sorting of data based on various attributes.

3.4.2.3.1 Query Parameters

- offset: An integer representing the starting point in the collection of records. The offset is zero-based, meaning an offset=0 refers to the first record.
  - Type: Integer
  - Default Value: 0
  - Example: offset=10
- limit: An integer representing the maximum number of records to return in the response.
  - Type: Integer
  - Default Value: 25
  - Example: limit=10

3.4.2.3.2 Example API Call

- **GET** [Base URL]/users?offset=10&limit=50

3.4.2.4 Cursor based Pagination

Cursor-based pagination allows clients to navigate through a dataset using a pointer (cursor) to the last item fetched, enabling more efficient querying, especially for large datasets. This method reduces the risk of missing or duplicating records when data changes during pagination. This method requires the data to be sorted. In cursor-based pagination, the cursor must be a unique attribute that can be sequentially ordered, and data will always be sorted by that attribute.

3.4.2.4.1 Query Parameters

- **cursor**: A unique identifier representing the position in the dataset. This value is typically derived from a specific attribute of the last item in the previous page (e.g., a timestamp or an ID).
  - Type: String
  - Default Value: None (used for the first page of results)
  - Example: cursor=eyJpZCI6MTAsInRpbW

- **limit:** An integer representing the maximum number of records to return in the response.
  - Type: Integer
  - Default Value: 25
  - Example: limit=50

3.4.2.4.2 Example API Call

GET [Base URL]/entitlements?cursor=eyJpZCI6MTAsInRpbW&limit=50

3.5 Error Codes & Error Handling

This section details the error codes that the API may return and provides guidance on how to handle these errors effectively. Understanding these error responses is crucial for developing robust and user-friendly applications.

3.5.1 HTTP Status Codes

The following table lists common HTTP status codes used by the API, along with their meanings and suggested handling strategies:

Status Code	Meaning	Description	Handling Recommendations
200 OK	Success	The request was successful, and the response body contains the requested data.	Process the response as needed.
201 Created	Success	The request was successful, and a new resource was created. The response body contains the details of the new resource.	Process the response and acknowledge the creation of the resource.
204 No Content	Success	The request was successful, but there is no content to return.	No action needed.
400 Bad Request	Client Error	The request could not be understood or was missing required parameters.	Check the request parameters and retry with valid data.
401 Unauthorized	Client Error	Authentication failed or user does not have permissions for the requested operation.	Verify credentials and confirm the user has the necessary permissions.
403 Forbidden	Client Error	The server understood the request but refused to authorize it.	Confirm user permissions and confirm access rights.
404 Not Found	Client Error	The requested resource could not be found.	Check the resource identifier and retry with a valid one.
405 Method Not Allowed	Client Error	The HTTP method used is not supported for the requested resource.	Verify the allowed methods and correct the request.
409 Conflict	Client Error	The request cannot be completed due to a conflict with the current state of the resource.	Resolve the conflict and retry the request.
422 Unprocessable Entity	Client Error	The request was well-formed but could not be processed due to semantic errors.	Correct the semantic errors and retry the request.
429 Too Many Requests	Client Error	The user has sent too many requests in a given amount of time.	Implement rate limiting and retry after a delay.
500 Internal Server Error	Server Error	The server encountered an unexpected condition that prevented it from fulfilling the request.	Retry the request later and report the issue if it persists.
502 Bad Gateway	Server Error	The server received an invalid response from an upstream server.	Retry the request later.

503 Service Unavailable	Server Error	The server is currently unable to handle the request due to maintenance or overload.	Retry the request later.
504 Gateway Timeout	Server Error	The server did not receive a timely response from an upstream server.	Retry the request later.

3.5.2 Error Response Structure

The API returns error responses in a standardized JSON format. This format typically includes an error code, message, and additional details if applicable.

```
{
  "error": {
    "code": 400,
    "message": "Invalid request parameters",
    "details": {
      "parameter": "email",
      "issue": "Email is required"
    }
  }
}
```

- code: A numerical code representing the error (aligns with the HTTP status code).
- message: A human-readable message describing the error.
- details: Additional information about the error to help with debugging (optional).

3.5.3 Handling Errors

- **Client-Side Validation**: Confirm that all required parameters and headers are included in the request and validate data types and formats before sending the request.
- **Authentication and Authorization**: Verify that the API request includes valid authentication tokens and that the user has the necessary permissions to perform the requested action.
- **Retry Mechanism**: Implement retry logic for transient errors (e.g., 500 Internal Server Error, 502 Bad Gateway). Use an exponential backoff strategy to prevent overwhelming the server.
- **Rate Limiting**: Respect the API's rate limits by implementing proper rate-limiting mechanisms and handling 429 Too Many Requests errors gracefully by retrying after the specified delay.
- **Logging and Monitoring**: Log error responses and monitor them to detect patterns or recurring issues. This information is useful for debugging and improving the API.
- **User Feedback**: Provide clear and actionable feedback to the users when errors occur. For example, if a 400 Bad Request error is returned, inform the user about the specific parameter that caused the issue.

4 REST API Implementation by Integration Type

In this section, the various supported HTTP Operations and the HTTP Methods that they can use are outlined in detail, for each management type for applications, i.e., Centrally, Locally or Hybrid Managed Apps. These are some of the usual IAM web services operations that need to be configured for IAM tools like SailPoint ISC to leverage the endpoints to build an IAM integration that serves CVS Health's requirements.

One way of approaching base URL construction is through a multi-level context. This approach uses a generic base URL, which multiple applications can have in common, but the context URL identifies what application the APIs are intended for, the versions and other specific endpoint details.

In such cases, a base URL to use in API calls looks like:

https://api.[domain].com/[application id]/[API version]/...

Another way of building out base URL structure is the one followed by **SaaS apps**:

https://[application id].api.[domain].com/[API version]/…

Anything within [square braces] shown above is a variable, and the suffix of a base URL usually is context-specific, which is detailed below.

4.1 Centrally Managed Applications

Summary of all the operations required for this integration type:

HTTP Operation	Required Operation	Reference Section
GET	**Get Entitlements**	Section 4.1.1

4.1.1 Get Centrally Managed Entitlements

4.1.1.1 General Information

Operation Description	The API endpoint allows for the retrieval and management of entitlement data from target application into MyAccess.
HTTP Method	GET
Context URL	[Base URL]/entitlements
Query Parameters	N/A
Authentication Mechanism	OAuth 2.0 (Preferred) Please refer to Section 2.5 for other authentication mechanisms.
Headers	**Attribute Name** / **Description** Authorization: Bearer token for authentication (Assuming OAuth 2.0 Authentication Mechanism) Content-Type: application/json Accept: application/json
Body	Not Required
Rate Limiting	For guidelines on Rate Limiting, refer to section 3.3

Pagination	For guidelines on Pagination, refer to section 3.4 The paginated result should be sorted by entitlement ID to ensure all the data is returned properly.
Error Handling	For guidelines on Error handling, refer to section 3.4
Error Structure	Every error should be returned in a JSON structure as outlined in section 3.4.2
Query Parameters	The following parameters can be considered to filter the results of this API call: • filter=entitlementType eq “Role” Consider filtering syntax with the following considerations: • <key> <operation> “<value>” • Here, key is the filter type • Operation can be in, eq, lt, gt, le, ge • Value is a string enclosed in quotes. **(Optional):** Only the following attributes are to be considered for filtering entitlements: 1. EntitlementType - ***eq*** 2. Environment - ***eq*** 3. Entitlement - ***eq***

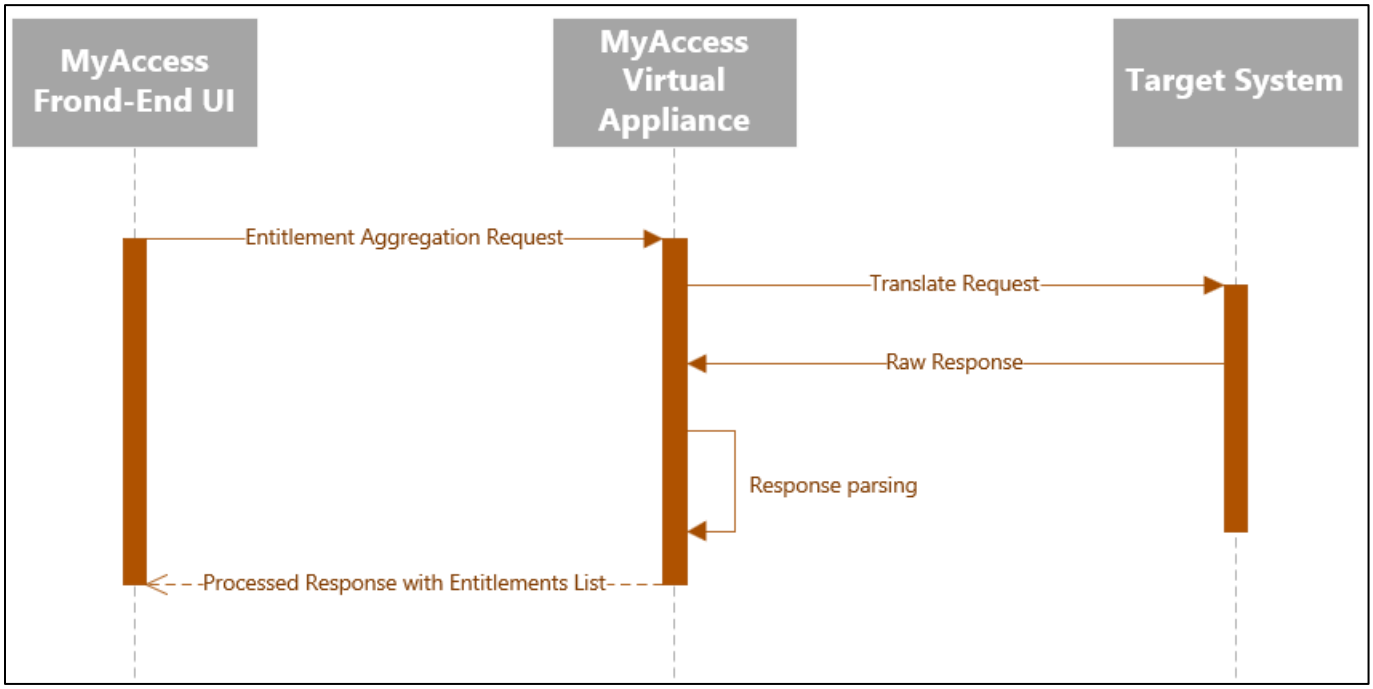
4.1.1.2 Response Information and Attributes

The following attributes can be included in the entitlement aggregation:

Attribute Name	Description	Optional vs Required
APMID	Unique identifier for the entitlement within the application **Type:** String **Example:** FBF421C35C6B7EF3	Required
Entitlement	Name of the entitlement, which typically indicates the permission or role granted **Type:** String **Example:** Administrator	Required
Environment	Specifies the context or environment in which the entitlement is applicable, such as development, testing, or production **Type:** String **Example:** PROD	Required
EntitlementType	Categorizes the nature of the entitlement, such as entitlement, group, permission, or access level, defining its function and scope within the system **Type:** String **Example:** Role	Required

4.1.1.3 Process Flow

- SailPoint sends a request to the REST API Web Service Connector.
- The connector translates the request and communicates with [Target System].
- [Target System] processes the request and returns a response.
- The connector translates the response back to SailPoint.



4.1.1.4 API Request & Response Illustration

4.1.1.4.1 Request Example:

- This operation does not require a request body.
  - Example of a request URL: [Base URL]/entitlement-list

For Base URL, refer to section 3.0, where a structure for Base URL has been identified.

4.1.1.4.2 Response Example (Centrally-Managed):

```
{
  "status": "success",
  "data": [
    {
      "APMID": "FBF421C35C6B7EF3",
      "Entitlement": "Admin User",
      "EntitlementType": "Group",
      "Environment": "Azure Prod"
    },
    {
      "APMID": "FBF421C35C6B7EE3",
      "Entitlement": "ADMIN01",
      "EntitlementType": "Group",
      "Environment": "TopSecret Prod"
    },
    {
      "APMID": "FBF421C35C6B8EA1",
      "Entitlement": "Base User",
      "EntitlementType": "Group",
      "Environment": "aeth.aetna.com"
    }
  ]
}
```


4.1.2 Get Entitlement Count

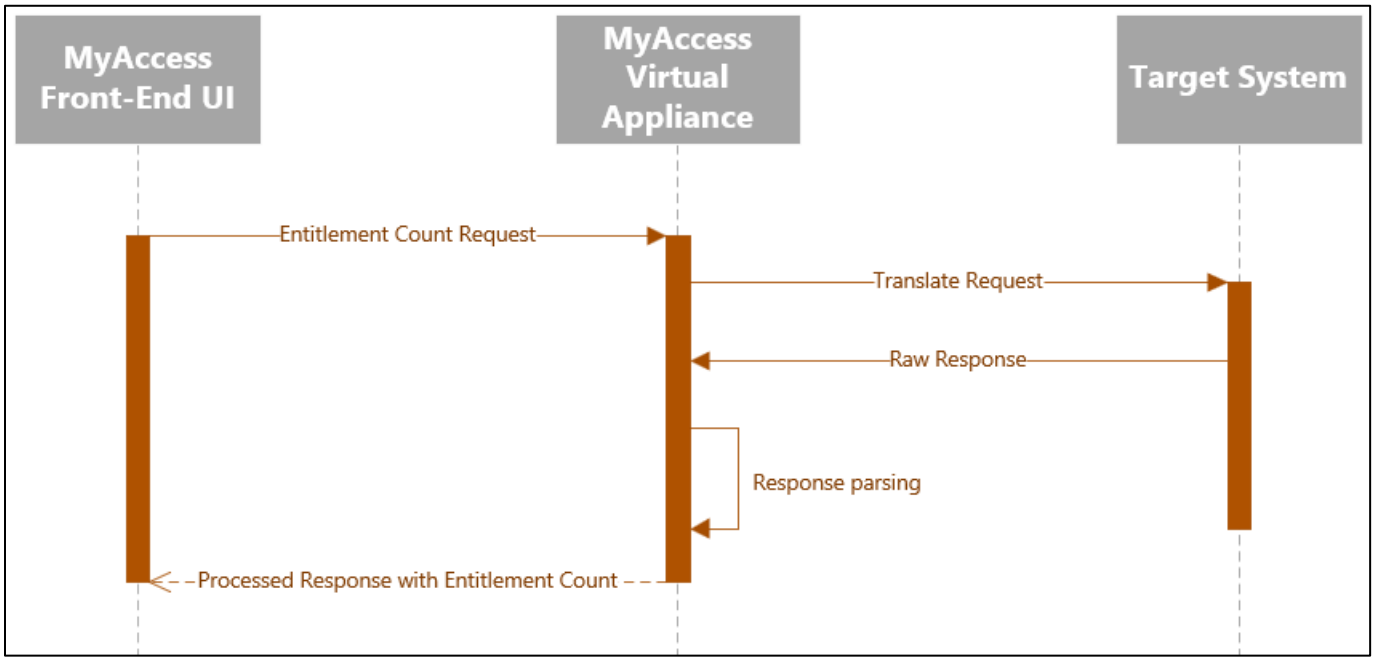
Operation Description	The Get Entitlement Count endpoint API allows for the retrieval of the total entitlements in the external system into SailPoint ISC. This API is usually used in assisting with pagination of users.
HTTP Method	GET
Context URL	[Base URL]/entitlements/count
Authentication Mechanism	OAuth 2.0 (Preferred) Please refer to Section 2.5 for other authentication mechanisms.
Headers	**Attribute Name** / **Description** Authorization — Bearer token for authentication Content-Type — application/json Accept — application/json
Body	Not Required
Rate Limiting	For guidelines on Rate Limiting, refer to section 3.3
Pagination	NA
Error Handling	For guidelines on Error handling, refer to section 3.5
Error Structure	Every error should be returned in a JSON structure as outlined in section 2.4.2
Query Parameters	NA

4.1.2.1 Response Information and Attributes

Attribute Name	Description	Required vs Optional
count	The total number of entitlements available for aggregation. **Type:** Integer **Example:** 544	Required

4.1.2.2 Process Flow

- SailPoint sends a request to the REST API Web Service Connector.
- The connector translates the request and communicates with [Target System].
- [Target System] processes the request and returns a response.
- The connector translates the response back to SailPoint.



4.1.2.3 API Request & Response Illustration

4.1.2.3.1 Request Example:

- This operation does not require a request body.
  - Example of a request URL: [Base URL]/entitlements/count
- For Base URL, refer to section 3.0, where a structure for Base URL has been identified.

4.1.2.3.2 Response Example:

```
{
  "status": "success",
  "count": 544
}
```

4.2 Locally Managed Applications

HTTP Operation	Required Operation	Reference Section
GET	Get Entitlements	Section 4.2.1.1
GET	Get All Users	Section 4.2.2.1
GET	Get Single User	Section 4.2.2.2
GET	Get User Count	Section 4.2.2.3
POST	Remove Membership	Section 4.2.2.4

4.2.1 Entitlement Management

4.2.1.1 Get Locally Managed Entitlements

4.2.1.1.1 General Information

Operation Description	An Entitlement Aggregation endpoint API allows for the retrieval and management of entitlement data from external systems into IAM tools like SailPoint ISC.
HTTP Method	GET
Context URL	[Base URL]/entitlements
Authentication Mechanism	OAuth 2.0 (Preferred) Please refer to Section 3.5 for other authentication mechanisms.
Headers	**Attribute Name** / **Description** Authorization — Bearer token for authentication Content-Type — application/json Accept — application/json
Body	Not Required
Rate Limiting	For guidelines on Rate Limiting, refer to section 3.2
Pagination	For guidelines on Pagination, refer to section 3.3 The paginated result should be sorted by entitlement ID to ensure all the data is returned properly.
Error Handling	For guidelines on Error handling, refer to section 3.4
Error Structure	Every error should be returned in a JSON structure as outlined in section 3.4.2

Query Parameters	The following parameters can be considered to filter the results of this API call: • filter=entitlementType eq “Role” Consider filtering syntax with the following considerations: • <key> <operation> “<value>” • Here, key is the filter type • Operation can be in, eq, lt, gt, le, ge • Value is a string enclosed in quotes. **(Optional):** The following attributes are to be considered for filtering entitlements: 1. EntitlementType - ***eq*** 2. Environment - ***eq*** 3. Entitlement - ***eq***

4.2.1.1.2 Response Information and Attributes

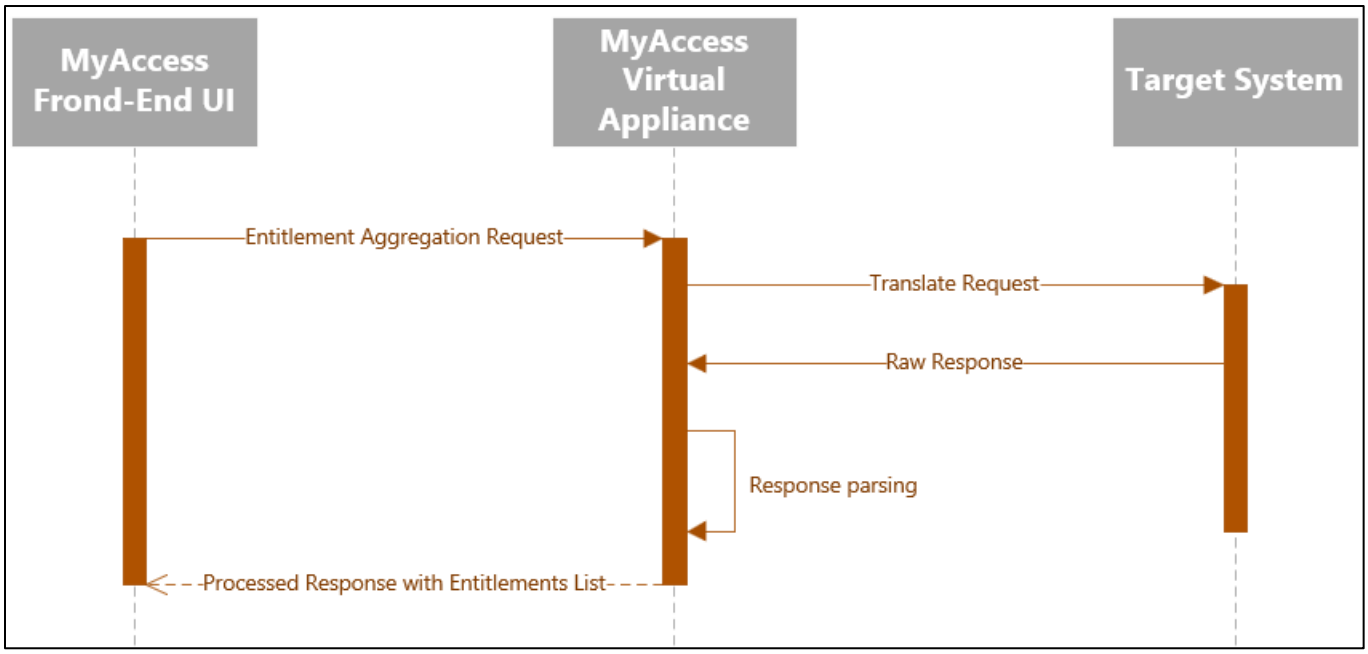
The following attributes can be included in the entitlement aggregation, but the list is non-exhaustive:

Attribute Name	Description	Required vs Optional
APMID	Unique identifier for the entitlement within the application **Type:** String **Example:** FBF421C35C6B7EF3	Required
Entitlement	Name of the entitlement, which typically indicates the permission or role granted **Type:** String **Example:** Administrator	Required
Environment	Specifies the context or environment in which the entitlement is applicable, such as development, testing, or production **Type:** String **Example:** PROD	Required
EntitlementDescription	Description of access granted by the Entitlement. Key fields to include in the description are <Application Name>, <Environment>, <type of access being granted i.e., read/edit/admin> & <components of the application to be listed where this entitlement grants access> Suggested description to be in the format of "This entitlement provides access to <Application Name> and <Environment>. The entitlement provides <read/edit/admin> privileges to <components of the application to be listed>". **Type:** String **Example:** This entitlement provides access to Azure and Production. The entitlement provides admin privileges to Azure Support Portal.	Required

EntitlementOwner1	"EntitlementOwner1" needs to be an employee and 7-digit CVS Employee Number. **Type:** String **Example**: C818671	Required
EntitlementOwner2	"EntitlementOwner2" needs to be an employee and 7-digit CVS Employee Number & cannot be same as Entitlement Owner 1. **Type:** String **Example:** C818001	Required
EntitlementType	Categorizes the nature of the entitlement, such as entitlement, group, permission, or access level, defining its function and scope within the system **Type:** String **Example:** Role	Required

4.2.1.1.3 Process Flow

- SailPoint sends a request to the REST API Web Service Connector.
- The connector translates the request and communicates with [Target System].
- [Target System] processes the request and returns a response.
- The connector translates the response back to SailPoint.



4.2.1.1.4 API Request & Response Illustration

4.2.1.1.4.1 Request Example:

- This operation does not require a request body.
  - Example of a request URL: [Base URL]/entitlement-list
- For Base URL, refer to section 3.0, where a structure for Base URL has been identified.

4.2.1.1.4.2 Response Example (Locally-Managed):

```
{
  "status": "success",
  "data": [
    {
      "APMID": "FBF421C35C6B7EF3",
      "Entitlement": "Admin Access",
      "EntitlementType": "Role",
      "Environment": "Prod",
      "EntitlementDescription": "Provides administrative access to the system",
      "EntitlementOwner1": "3244231",
      "EntitlementOwner2": "3244232"
    },
    {
      "APMID": "FBF421C35C6B7EF3",
      "Entitlement": "Auditor Access",
      "EntitlementType": "Role",
      "Environment": "Prod",
      "EntitlementDescription": "Provides user-level access to the system",
      "EntitlementOwner1": "3244242",
      "EntitlementOwner2": "3244249"
    }
  ]
}
```


4.2.1.2 Get Entitlement Count

<table>
<tr><td>Operation Description</td><td>The Get Entitlement Count endpoint API allows for the retrieval of the total entitlements in the external system into SailPoint ISC. This API is usually used in assisting with pagination of users.
Note: This count should match the total number of entitlements being retrieved by the GetEntitlements API.</td></tr>
<tr><td>HTTP Method</td><td>GET</td></tr>
<tr><td>Context URL</td><td>[Base URL]/entitlements/count</td></tr>
<tr><td>Authentication Mechanism</td><td>OAuth 2.0 (Preferred)
Please refer to Section 2.5 for other authentication mechanisms.</td></tr>
<tr><td>Headers</td><td>
<table>
<tr><th>Attribute Name</th><th>Description</th></tr>
<tr><td>Authorization</td><td>Bearer token for authentication</td></tr>
<tr><td>Content-Type</td><td>application/json</td></tr>
<tr><td>Accept</td><td>application/json</td></tr>
</table>
</td></tr>
<tr><td>Body</td><td>Not Required</td></tr>
<tr><td>Rate Limiting</td><td>For guidelines on Rate Limiting, refer to section 3.3</td></tr>
<tr><td>Pagination</td><td>NA</td></tr>
<tr><td>Error Handling</td><td>For guidelines on Error handling, refer to section 3.5</td></tr>
<tr><td>Error Structure</td><td>Every error should be returned in a JSON structure as outlined in section 2.4.2</td></tr>
</table>

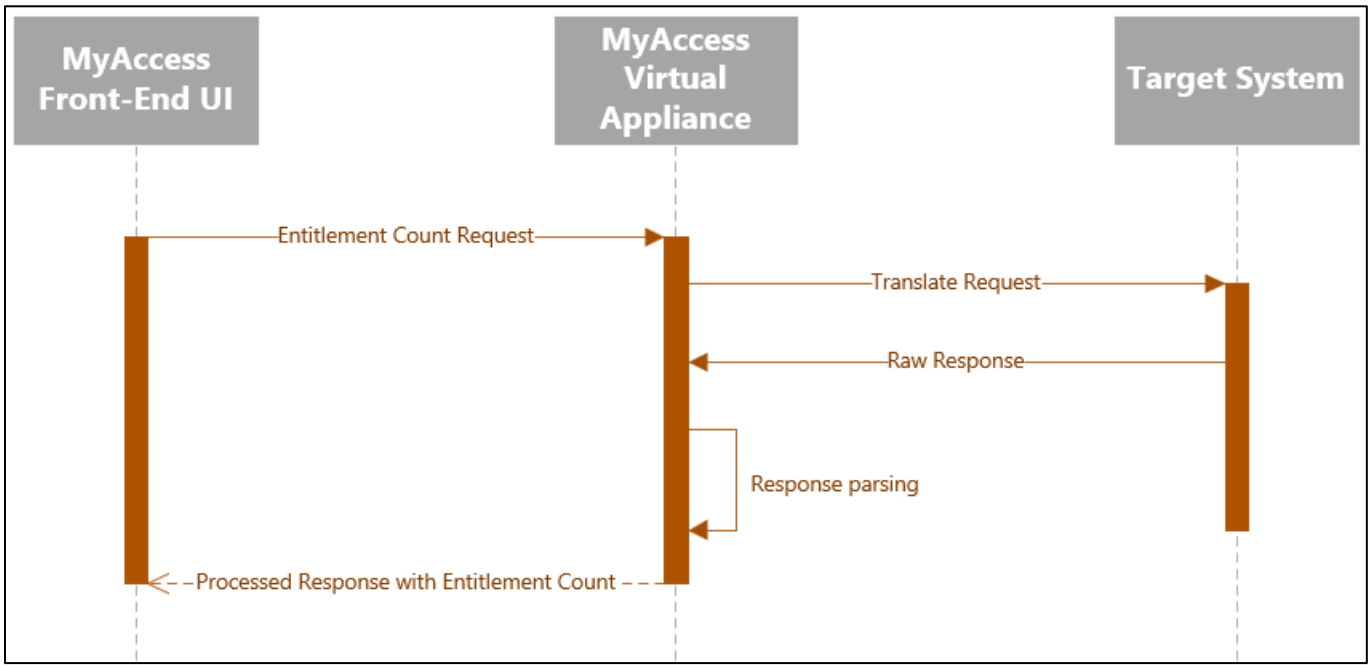
Query Parameters	NA

4.2.1.2.1 Response Information and Attributes

Attribute Name	Description	Required vs Optional
count	The total number of entitlements available for aggregation. **Type:** Integer **Example:** 544	Required

4.2.1.2.2 Process Flow

- SailPoint sends a request to the REST API Web Service Connector.
- The connector translates the request and communicates with [Target System].
- [Target System] processes the request and returns a response.
- The connector translates the response back to SailPoint.



4.2.1.2.3 API Request & Response Illustration

4.2.1.2.3.1 Request Example:

- This operation does not require a request body.
  - Example of a request URL: [Base URL]/entitlements/count
- For Base URL, refer to section 3.0, where a structure for Base URL has been identified.

4.2.1.2.3.2 Response Example:

```
{
  "status": "success",
  "count": 544
}
```

4.2.2 Account Management

4.2.2.1 Get User Membership

<table>
<tr><td>Operation Description</td><td>The Account Aggregation endpoint API allows for the retrieval and management of user account data from external systems into SailPoint ISC.</td></tr>
<tr><td>HTTP Method</td><td>GET</td></tr>
<tr><td>Context URL</td><td>[Base URL]/users</td></tr>
<tr><td>Authentication Mechanism</td><td>OAuth 2.0 (Preferred)
Please refer to Section 3.5 for other authentication mechanisms.</td></tr>
<tr><td>Headers</td><td>
<table>
<tr><th>Attribute Name</th><th>Description</th></tr>
<tr><td>Authorization</td><td>Bearer token for authentication</td></tr>
<tr><td>Content-Type</td><td>application/json</td></tr>
<tr><td>Accept</td><td>application/json</td></tr>
</table>
</td></tr>
<tr><td>Body</td><td>Not Required</td></tr>
<tr><td>Rate Limiting</td><td>For guidelines on Rate Limiting, refer to section 3.2</td></tr>
<tr><td>Pagination</td><td>For guidelines on Pagination, refer to section 3.3
The paginated result should be sorted by account ID to ensure all the data is returned properly.</td></tr>
<tr><td>Error Handling</td><td>For guidelines on Error handling, refer to section 3.4</td></tr>
<tr><td>Error Structure</td><td>Every error should be returned in a JSON structure as outlined in section 3.4.2</td></tr>
<tr><td>Query Parameters</td><td>The following parameters can be considered to filter the results of this API call:
• filter=accountType eq “Privileged”
Consider filtering syntax with the following considerations:
• <key> <operation> “<value>”
• Here, key is the filter type
• Operation can be in, eq, lt, gt, le, ge
• Value is a string enclosed in quotes.
(Optional): The following attributes are to be considered for filtering accounts:
1. IDType - eq
2. accountId - eq
3. employeeNumber - eq
4. ownerID - eq
5. AccountStatus - eq
6. Modified – gt, lt</td></tr>
</table>

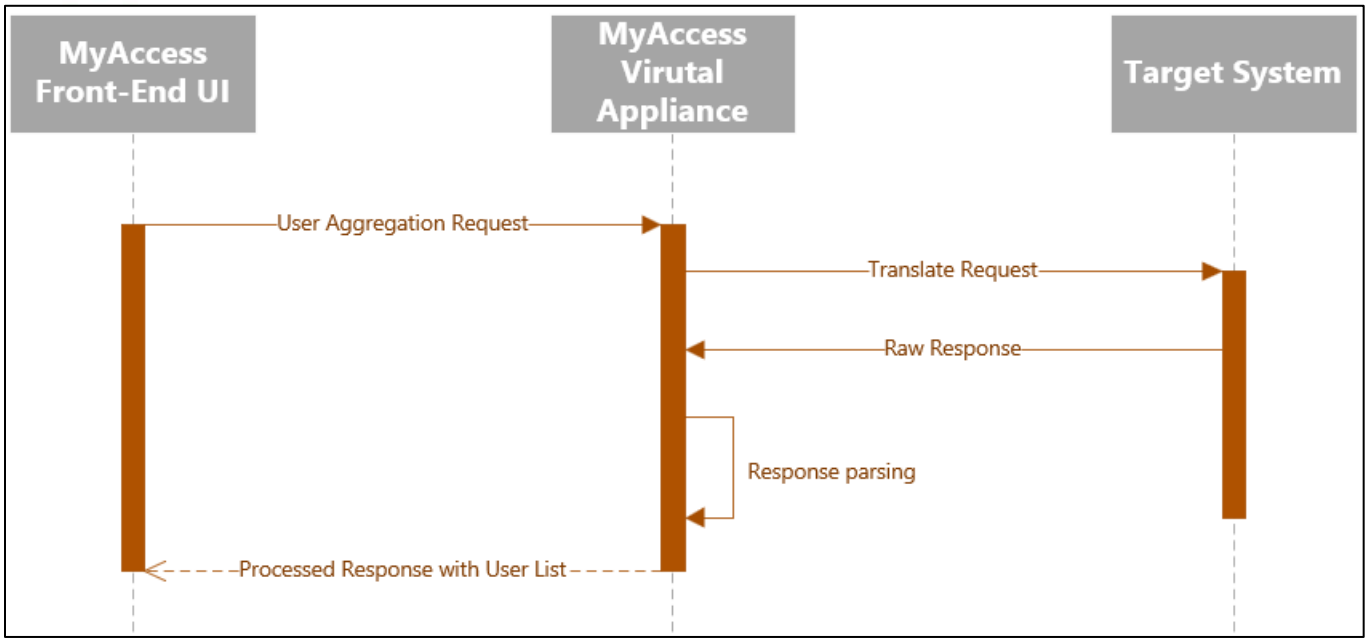
4.2.2.1.1 Response Information and Attributes

The following attributes can be included in the user aggregation, but this list is non-exhaustive.

Attribute Name	Description	Required vs Optional
IDType	Type of account: Person, Non-Person, External (client/third party) **Type:** String **Example:** Person	**Required**
accountId	Native account ID in the system **Type:** String **Example:** 3244231	**Required**
employeeNumber	7-digit CVS employee number of the user of this account **Type:** String **Example:** C818671	**Required** – If IDType is “Person” **Optional** – If IDType is “Non-Person” or “External”
ownerID	7-digit CVS employee number of the owner of this account **Type:** String **Example**: C818671	**Required** – If IDType is “Non-Person” or “External” **Optional** – If IDType is “Person”
FirstName	First name of the user account **Type:** String **Example**: John	**Optional**
LastName	Last name of the user account **Type:** String **Example**: Doe	**Optional**
Descriptor	Description of the account **Type:** String **Example**: Created by SailPoint on 10/03/2022 through automated provisioning.	**Required** – If IDType is “Non-Person” or “External” **Optional** – If IDType is “Person”
AccountStatus	Status of account: active vs inactive **Type:** String **Example:** active	**Required**
Entitlement	Entitlement that account has access to **Type:** Array **Example**: Read, Write, Execute	**Required**
Modified (Optional)	The last modified timestamp, ideally in format yyyy-MM-dd'T'kk:mm:ss'Z' **Type**: String	**Optional**
Email (Optional)	The email of the user. **Type**: String **Example**: john.doe@cvshealth.com	**Optional**

4.2.2.1.2 Process Flow

- SailPoint sends a request to the REST API Web Service Connector.
- The connector translates the request and communicates with [Target System].
- [Target System] processes the request and returns a response.
- The connector translates the response back to SailPoint.



4.2.2.1.3 API Request & Response Illustration

4.2.2.1.3.1 Request Example

- This operation does not require a request body.
  - Example of a request URL: [Base URL]/user-list
- For Base URL, refer to section 3.0, where a structure for Base URL has been identified.

4.2.2.1.3.2 Response Example (Locally-Managed):

```
{
  "status": "success",
  "count": 2,
  "data": [
    {
      "IDType": "Person",
      "accountId": " C818671",
      "employeeNumber": "3244211",
      "ownerId": "",
      "firstName": "John",
      "lastName": "Doe",
      "Descriptor": "",
      "AccountStatus": "active",
      "Email": "jdoe@example.com ",
      "Modified": "2023-10-05T15:30:45Z",
      "Entitlement": [
        "Admin",
        "User"
      ]
    },
    {
      "IDType": "Non-Person",
      "accountId": "C818321",
      "ownerId": " 3244212",
      "Descriptor": "Service Account Used by Oracle Vendor Applications",
      "AccountStatus": "active",
      "Email": "oracle.services@example.com ",
      "Modified": "2023-10-05T15:45:45Z",
      "Entitlement": [
        "View",
        "Modify",
        "RunTask"
      ]
    }
  ]
}
```


4.2.2.2 Get User

Operation Description	The Single Account Aggregation endpoint API allows for the retrieval and management of a specific user's account data from the external system into SailPoint ISC.
HTTP Method	GET
Context URL	[Base URL]/users/[accountId]
Authentication Mechanism	OAuth 2.0 (Preferred) Please refer to Section 3.5 for other authentication mechanisms.
Headers	**Attribute Name** / **Description** Authorization — Bearer token for authentication Content-Type — application/json Accept — application/json

Body	Not Required
Rate Limiting	For guidelines on Rate Limiting, refer to section 3.2
Pagination	NA
Error Handling	For guidelines on Error handling, refer to section 3.5 For Get User API, the following specific errors may need to be handled: • 404 Not Found – Account not found with a given account ID
Error Structure	Every error should be returned in a JSON structure as outlined in section 3.4.2
Query Parameters	NA

4.2.2.2.1 Response Information and Attributes

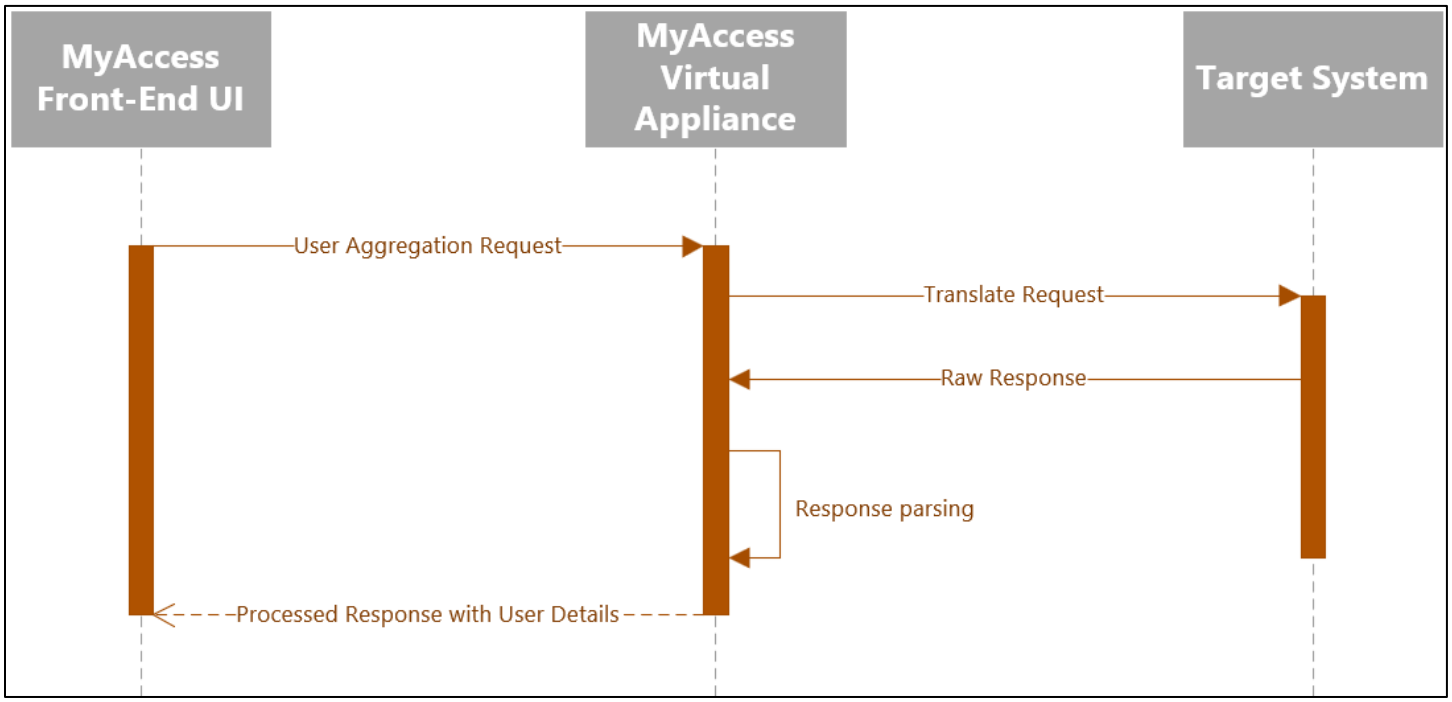
The following attributes can be included in the user aggregation, but this list is non-exhaustive.

Attribute Name	Description	Required vs Optional
IDType	Type of account: Person, Non-Person, External (client/third party) **Type:** String **Example:** Person	**Required**
accountId	Native account ID in the system **Type:** String **Example:** C818671	**Required**
employeeNumber	7-digit CVS employee number of the user of this account **Type:** String **Example:** 3244546	**Required** – If IDType is "Person" **Optional** – If IDType is "Non-Person" or "External"
ownerID	7-digit CVS employee number of the owner of this account **Type:** String **Example**: 3244546	**Required** – If IDType is "Non-Person" or "External" **Optional** – If IDType is "Person"
FirstName	First name of the user account **Type:** String **Example**: John	**Optional**
LastName	Last name of the user account **Type:** String **Example**: Doe	**Optional**
Descriptor	Description of the account **Type:** String **Example**: Created by SailPoint on 10/03/2022 through automated provisioning.	**Required** – If IDType is "Non-Person" or "External" **Optional** – If IDType is "Person"
AccountStatus	Status of account: active vs inactive **Type:** String **Example:** active	**Required**
Entitlement	Entitlements that account has access to **Type:** Array **Example**: Read, Write, Execute	**Required**

Email (Optional)	The email of the user. **Type**: String **Example**: john.doe@cvshealth.com	**Optional**
		Optional

4.2.2.2.2 Process Flow

- SailPoint sends a request to the REST API Web Service Connector.
- The connector translates the request and communicates with [Target System].
- [Target System] processes the request and returns a response.
- The connector translates the response back to SailPoint.



4.2.2.2.3 API Request & Response Illustration

4.2.2.2.3.1 Request Example

- This operation does not require a request body.
  - Example of a request URL: [Base URL]/users/C818671
- For Base URL, refer to section 3.0, where a structure for Base URL has been identified.

4.2.2.2.3.2 Response Example

```
{
  "status": "success",
  "count": 1,
  "data": [
    {
      "IDType": "Person",
      "accountId": " C818671",
      "employeeNumber": "3244567",
      "ownerId": "",
      "firstName": "John",
      "lastName": "Doe",
      "Descriptor": "",
      "AccountStatus": "active",
      "Email": "jdoe@example.com ",
      "Modified": "2023-10-05T15:30:45Z",
      "Entitlement": [
        "Admin",
        "User"
      ]
    }
  ]
}
```


4.2.2.3 Get User Count

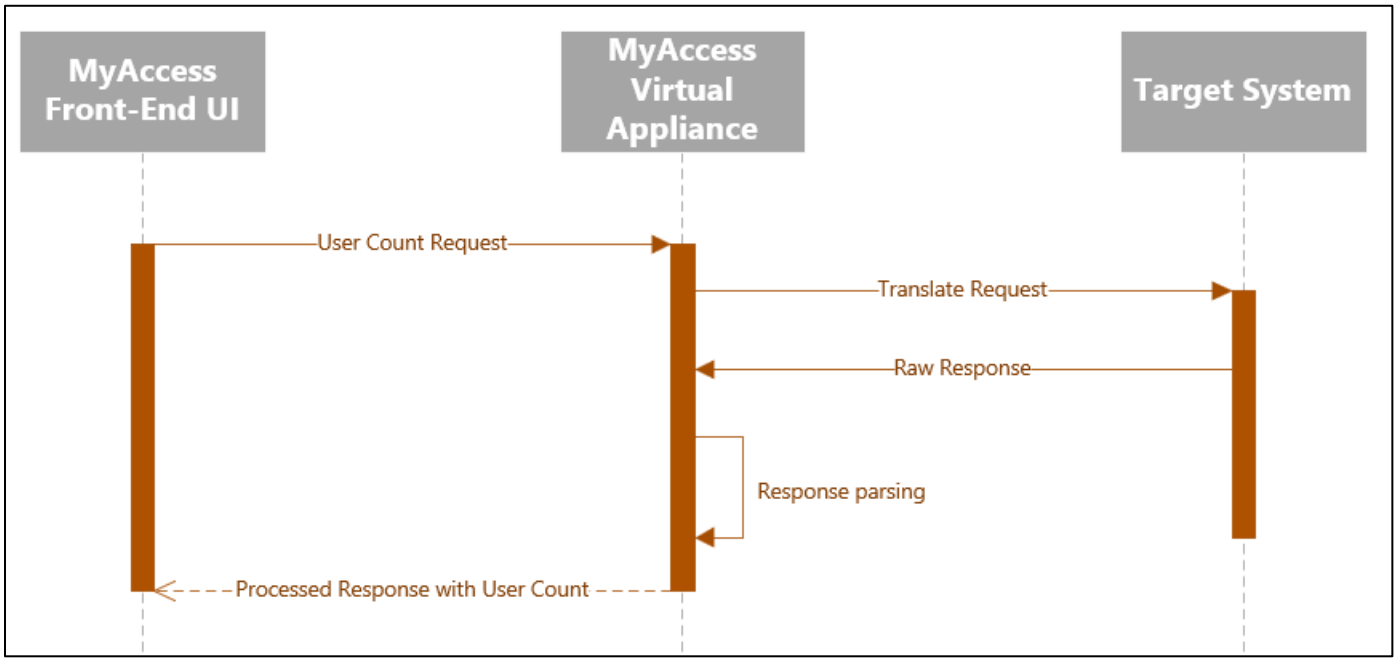
Operation Description	The Get User Count endpoint API allows for the retrieval of the total users in the external system into SailPoint ISC. This API is usually used in assisting with pagination of users. **Note:** The count should match the total number of users being retrieved at the time of the request being made.
HTTP Method	GET
Context URL	[Base URL]/users/count
Authentication Mechanism	OAuth 2.0 (Preferred) Please refer to Section 2.5 for other authentication mechanisms.
Headers	**Attribute Name** / **Description** Authorization — Bearer token for authentication Content-Type — application/json Accept — application/json
Body	Not Required
Rate Limiting	For guidelines on Rate Limiting, refer to section 3.3
Pagination	NA
Error Handling	For guidelines on Error handling, refer to section 3.5
Error Structure	Every error should be returned in a JSON structure as outlined in section 2.4.2
Query Parameters	NA

4.2.2.3.1 Response Information and Attributes

Attribute Name	Description	Required vs Optional
count	The total number of users available for aggregation. **Type:** Integer **Example:** 1347	Required

4.2.2.3.2 Process Flow

- SailPoint sends a request to the REST API Web Service Connector.
- The connector translates the request and communicates with [Target System].
- [Target System] processes the request and returns a response.
- The connector translates the response back to SailPoint.



4.2.2.3.3 API Request & Response Illustration

4.2.2.3.3.1 Request Example:

- This operation does not require a request body.
  - Example of a request URL: [Base URL]/users/count
- For Base URL, refer to section 3.0, where a structure for Base URL has been identified.

4.2.2.3.3.2 Response Example:

```
{
  "status": "success",
  "count": 1347
}
```

4.2.2.4 Remove Entitlement

<table>
<tr><td>Operation Description</td><td>The Remove Entitlement endpoint API allows for revoking of a specific entitlement from a user. It returns a status code presenting the success or failure of the API operation for a client like SailPoint ISC to process the downstream actions to be taken. This API is called for a single account and can be used to revoke one or more entitlements.</td></tr>
<tr><td>HTTP Method</td><td>POST</td></tr>
<tr><td>Context URL</td><td>[Base URL]/user/[accountId]/change-access</td></tr>
<tr><td>Authentication Mechanism</td><td>OAuth 2.0 (Preferred)
Please refer to Section 3.5 for other authentication mechanisms.</td></tr>
<tr><td>Headers</td><td>
<table>
<tr><th>Attribute Name</th><th>Description</th></tr>
<tr><td>Authorization</td><td>Bearer token for authentication</td></tr>
<tr><td>Content-Type</td><td>application/json</td></tr>
<tr><td>Accept</td><td>application/json</td></tr>
</table>
</td></tr>
<tr><td>Body</td><td>

```
{
    "entitlementType":"Role",
    "data":[
        {
            "operation": "Remove",
            "entitlementName":"Admin",
            "APMID":" FBF421C35C6B7EF3"
        },
        {
            "operation": "Remove",
            "entitlementName":"Auditor",
            "APMID":" FBF421C35C6B7EF3"
        }
    ]
}
```

</td></tr>
<tr><td>Rate Limiting</td><td>For guidelines on Rate Limiting, refer to Section 3.2</td></tr>
<tr><td>Pagination</td><td>NA</td></tr>
<tr><td>Error Handling</td><td>For guidelines on Error handling, refer to Section 3.4
<table>
<tr><th>Response Code</th><th>Scenario</th></tr>
<tr><td>200 (OK)</td><td>Successful Revocation</td></tr>
<tr><td>404 (Not Found)</td><td>User does not exist in the application
OR
Entitlement does not exist in the application</td></tr>
<tr><td>400 (Bad Request)</td><td>Invalid JSON Payload</td></tr>
<tr><td>422 (Unprocessable Entity)</td><td>User does not have entitlement provided in the JSON request payload</td></tr>
</table>
</td></tr>
</table>

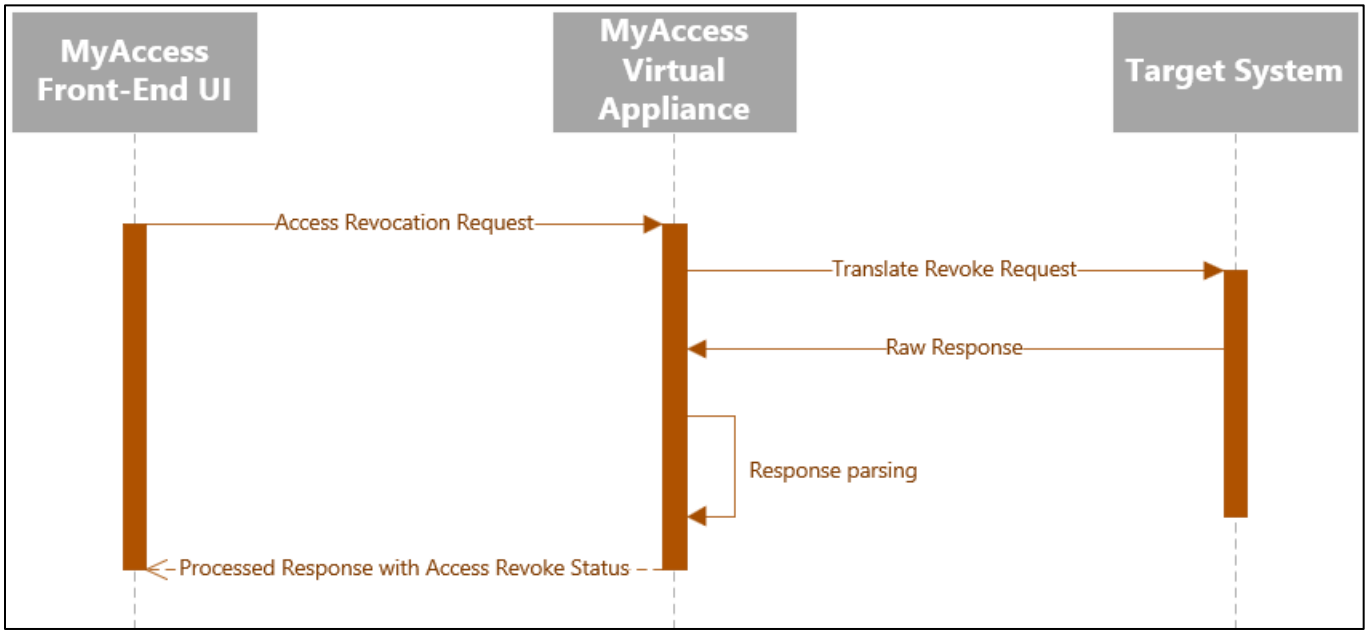
	500 (Internal Server Error)	Server-side error during revocation of entitlements provided in the JSON request payload Note: SailPoint will retry and optimize the requests contextually.
Error Structure	Every error should be returned in a JSON structure as outlined in Section 3.4.2	
Query Parameters	NA	

4.2.2.4.1 Response Information and Attributes

This API does not provide a response body. Just an HTTP code of 200 OK for successful revocation and error codes otherwise.

4.2.2.4.2 Process Flow

- SailPoint sends a request to the REST API Web Service Connector.
- The connector translates the request and communicates with [Target System].
- [Target System] processes the request and returns a response.
- The connector translates the response back to SailPoint.



4.2.2.4.3 API Request & Response Illustration

4.2.2.4.3.1 Request Example:

- Example of a request URL: [Base URL]/user/[accountId]/change-access

- For Base URL, refer to section 3.0, where a structure for Base URL has been identified.

4.2.2.4.3.2 Response Example:

```
{
    "status":"Success"
}
```

4.3 Hybrid Managed Applications

The way REST APIs for hybrid managed applications can be implemented is very similar to the descriptions outlined for the locally managed applications' REST API operations. As such, you can refer to the same sections above to dive into further details. The sections for different operations to implement have been linked in the table below:

HTTP Operation	Required Operation	Reference Section
GET	**Get Entitlements**	Section 4.2.1.1
GET	**Get Entitlement Count**	Section 4.2.1.2
GET	**Get All Users**	Section 4.2.2.1
GET	**Get Single User**	Section 4.2.2.2
GET	**Get User Count**	Section 4.2.2.3
POST	**Remove Membership**	Section 4.2.2.4